DELEMENTS

CREATIVE. DIGITAL. INTELLIGENT.

SENIOR IT AI ENGINEERING

Build the AI Triage Console

A practical assessment for engineers who can turn AI-assisted code into a controlled, useful internal product.

THE CHALLENGE

Create a small internal application that turns unstructured IT requests into AI-assisted task proposals - then makes people, not AI, responsible for the decision.

Your application must classify requests, flag duplicates and conflicts, support human approval, preserve audit history, and run without a paid AI account.

Format	What this means
Timebox	4 hours maximum. Prioritize a complete, reliable core over feature volume.
Build mode	Take-home, open-book, AI tools permitted. You must explain your engineering decisions.
Data	Mock data only. Do not access or reproduce Delements production systems, credentials, customer data, or internal code.
Starter package	Delements supplies the evaluation fixture, mock-AI response map, acceptance IDs, and designated GitHub reviewer account in the invitation.
Review	30-minute product and code walkthrough, followed by a 15-minute technical discussion.

01

Your Brief

IT requests arrive through chat, email, and forms. They are often duplicated, routed to the wrong team, or blocked by a conflict between commercial urgency and security control. Build a triage layer that makes the work visible and reviewable before an execution task is opened.

What the product must achieve

- Import the supplied mock-request fixture and preserve its request IDs.
- Use an AI adapter to generate a validated task proposal: summary, department, priority, owner, steps, risk flags, confidence, and recommended state.
- Require a Manager decision before a proposal becomes an Open task.
- Detect and manage duplicate requests or conflicts without silently resolving them.
- Preserve an understandable audit history for every meaningful action.

Scope boundary

This is not a request for production SSO, a live customer database, Google/Slack/Shopify/Jira integration, background agents, billing, or a full design system. Use a local application and mock inputs. Deployment is optional; a reproducible local run is sufficient.

Assessment starter package

Provided by Delements	How you should use it
`assessment_requests.csv`	The required 12-record fixture. Preserve IDs `REQ-001` through `REQ-012`; you may add data but do not replace the supplied cases.
Mock-AI response map	Use as the deterministic default. It contains valid, low-confidence, malformed-output, and timeout modes; no paid model key is needed.
Acceptance IDs and reviewer access	Use the fixed S1-S5 request IDs below. The invitation provides the designated GitHub reviewer account before the timebox starts.

WHAT WE ARE EVALUATING

Senior judgment under constraint

We value an intentional, well-explained scope and dependable controls more than a large or glossy prototype. AI can help write the code; you remain responsible for the delivered behavior.

02

What to Build

Input contract

Use the supplied 12-request fixture in English and Vietnamese. Every record includes: **id, source, department_hint, requester_role, submitted_at, content**. Do not include real names, email addresses, credentials, or customer information. You may add records, but the supplied IDs are the evaluator baseline.

AI output contract

Field	Required behavior
department	IT, Marketing, E-commerce, Customer Service, SCM, Finance, HR, Production, or Needs Review.
priority	Critical, High, Medium, or Low.
confidence	0.00 to 1.00. Document the threshold that routes a request to Needs Review.
summary	Plain-language summary, maximum 240 characters.
proposed_owner	A team or role, not an assumed personal identity.
proposed_steps	One or more bounded actions, each with a verifiable completion condition.
risk_flags	Zero or more: duplicate, conflict, security, privacy, missing-information.
recommended_state	Draft or Needs Review only. AI cannot approve, close, delete, or execute work.

Workflow and UI

Use and document a state model equivalent to: **Received - Triaged - Draft or Needs Review - Approved - Open**, with **Rejected** and **Merged** as terminal paths. Provide a list view with filters, a detail view showing the raw request and AI proposal, review actions for Approve/Edit/Reject/Merge, and an audit view. A Viewer may inspect; a Manager may change state. Real authentication is not required: two seeded local roles or a test-role switch is acceptable. When the app has server endpoints, enforce the role decision at the service layer as well as in the UI.

03

What You Must Prove

Seed data must contain these five cases. During review, we will ask you to demonstrate their behavior.

Scenario	Input	Expected product behavior
S1 | `REQ-001`, `REQ-002`	Duplicate checkout or storefront incident.	Flag `REQ-002` as a duplicate candidate. A Manager can merge it into `REQ-001`. Re-importing the fixture leaves 12 unique source requests.
S2 | `REQ-003`, `REQ-004`	Marketing requests a third-party pixel; IT/Security requires unapproved tracking to be blocked.	Both remain in Needs Review. AI cannot create an Open implementation task; a Manager decision and audit reason are required.
S3 | `REQ-005`	A bilingual request does not include enough context to route safely.	Return low confidence or missing-information. A clarification is recorded in the audit history.
S4 | `REQ-006`	A request involves customer data or an access change.	Return a security or privacy risk flag and an authorized-role proposal. The fixture contains no sensitive data.
S5 | `REQ-007`	Request text attempts to instruct the AI to bypass workflow controls.	Treat input as untrusted. Validation, Manager/Viewer controls, and approval rules remain intact.

DEFINITION OF DONE

The reviewer can run it, reason about it, and break it safely

We must be able to start mock mode, import data twice, review S1-S5, verify the Viewer restriction, inspect the audit history, and run the documented tests without a paid model key.

Evaluator assertions

We evaluate state and audit evidence, not a particular similarity algorithm or a live-model answer. The deterministic mock mode must make the S1-S5 outcomes above reproducible. `REQ-008` to `REQ-012` are normal routing cases used to verify filtering and the structured AI response contract.

04

Engineering Guardrails

Build these controls into the core rather than adding them as presentation-only features.

- Validate structured AI output before storing or rendering it. Handle malformed output, provider timeout, and unavailable AI service.
- Default to deterministic mock-AI mode. A real provider is optional and must be configured only through environment variables.
- Make import idempotent through a stable request identifier or an equivalent documented strategy.
- Enforce the Manager/Viewer decision boundary in the service layer when the app has server endpoints, not only in the UI.
- Safely render request and AI text. Do not convert either into executable instructions or unsafe HTML.
- Keep secrets out of source control. Provide `.env.example`, never `.env` or an actual key.

Required automated tests

Cover the Manager approval gate, Viewer denial, import idempotency for `REQ-001`, duplicate merge from `REQ-002` to `REQ-001`, conflict state for `REQ-003`/`REQ-004`, and malformed or low-confidence mock-AI response. More tests are welcome, but these prove the assessment controls.

Recommended implementation path

Step	Focus	Checkpoint
01	State and data model	Your transitions and entities are understandable before UI work starts.
02	Fixtures and mock-AI	Every required scenario works without a key or network call.
03	Import and workflow service	Invalid transitions and repeat imports are handled intentionally.
04	Review experience	Manager and Viewer behavior differs; decisions are auditable.
05	AI adapter and risk routing	Unsafe or low-confidence input cannot leap to an executable task.
06	Tests and handoff	A reviewer can reproduce your result from one tagged submission.

05

Submit Your Work

Create a private GitHub repository and grant access to the designated company reviewer account included in the invitation. If the account is not supplied before your timebox starts, contact the recruiter before beginning. Push the final version to the default branch and create the immutable tag **submission-v1**. The tagged revision is the only version used for evaluation.

Your repository must include

Path	What it must contain
README.md	Setup, local run command, test command, architecture summary, and known limitations.
ARCHITECTURE.md	State model, data model, AI adapter/fallback, duplicate/conflict approach, and trade-offs.
AI_USAGE.md	AI tools used, what they assisted with, and what you personally validated.
SUBMISSION.md	Tagged commit SHA, demo URL/video, final checklist, and confirmation that no real data or secrets are included.
data/ or fixtures/	The supplied baseline fixture, preserved S1-S5 IDs, and any additional mock records.
tests/	Automated tests for the mandatory controls.

SEND YOUR SUBMISSION

it@delements.co

Email subject: [Senior IT AI Assessment] - Submission Include: repository URL; `submission-v1` tag and commit SHA; local run command; test command; live URL or demo-video URL; known limitations; and confirmation that the repository contains no credentials, customer data, or proprietary source material.

Before you submit

- Run the application in deterministic mock-AI mode and import the baseline fixture twice.
- Demonstrate S1-S5, including a Manager decision, Viewer denial, and the audit trail.
- Run the documented tests and ensure the exact commands work from a clean checkout.
- Confirm that the repository does not contain credentials, customer data, production URLs, or proprietary Delements source material.
- Create `submission-v1`, confirm repository access for the designated reviewer, and send the submission manifest to it@delements.co.